LANGGRAPH 0.3+ • OBSERVABILITY & TELEMETRY

Agentic AI Observability: Debugging & Monitoring with LangSmith

Non-Blocking Telemetry, Call-Tree Visualization, Latency & Token Analytics, and ReAct Execution Tracing

An authoritative technical deep dive into observability and runtime monitoring for agentic state machines. Deconstructs how LangSmith intercepts graph transitions, records full input/output state histories, visualizes nested multi-hop tool execution trees, and tracks production latency and token consumption without adding overhead to the user application.

Target Notebook: `4-Debugging&Monitoring.ipynb`

Generation Date: November 01, 2026

Language & Runtime: Python 3.14 • Virtualenv (.venv)

Core Frameworks: LangGraph 0.3+ • LangChain 1.4.7 • LangSmith • Groq LPU

B. Learning Objectives

After studying this notebook and study guide, you will master:

- **The Observability Imperative:** Understand why black-box LLM agents require end-to-end distributed tracing to diagnose hallucinations, recursive tool loops, and latency spikes.
- **LangSmith Native Tracing:** Enable automated lifecycle telemetry across LangChain and LangGraph using `LANGSMITH_TRACING="true"` without manual logging or code rewrites.
- **Decoupled Asynchronous Telemetry:** Understand how the LangSmith client buffers traces in an in-memory queue and flushes them in a background daemon thread with zero impact on agent response times.
- **Hierarchical Run Tree Deconstruction:** Read and interpret nested LangSmith trace trees, navigating from root `LangGraph` invocations down to node execution, model calls, and individual tool runs.
- **Linear vs. Cyclical Trace Signatures:** Contrast the single-step telemetry of direct conversational answers with the multi-hop trace signature of tool-augmented ReAct cycles.
- **Environment Variable Hygiene:** Correctly configure `LANGSMITH_PROJECT`, `LANGSMITH_API_KEY`, and diagnose common configuration pitfalls (such as project naming typos).

C. Prerequisites, Environment & Security Setup

> **Runtime & Telemetry Configuration**
> This notebook operates on **Python 3.14**. It utilizes the high-throughput **Groq LPU** running `openai/gpt-oss-20b`, orchestrated by **LangGraph**, and telemetrically instrumented via **LangSmith**.

Component	Version / Identifier	Source / Purpose
`langsmith`	`>=0.14.12`	Official observability SDK, asynchronous trace queue, OpenTelemetry exporter
`langgraph`	`>=0.3.0`	State machine execution engine, `StateGraph`, `ToolNode`
`langchain`	`1.4.7`	Core model abstractions, `init_chat_model`, message schemas
`langchain-groq`	`>=1.1.3`	Ultra-fast LPU inference engine with native tool calling support
`python-dotenv`	`>=1.2.4`	Secure environment variable loading from local `.env` file

> **Security & API Key Masking Protocol**
> Never commit LangSmith API keys or LLM provider keys to Git. Keep credentials isolated in `.env`:

```
GROQ_API_KEY=gsk_your_groq_api_key_here
LANGCHAIN_API_KEY=lsv2_pt_your_langsmith_api_key_here
LANGSMITH_TRACING=true
LANGSMITH_PROJECT=TestProject
```

D. Comprehensive Topic Overview

When building multi-actor agentic workflows, debugging via traditional `print()` statements quickly collapses. An agent may execute multiple model inferences, invoke various external tools, format and inspect JSON payloads, and loop conditionally before returning an answer. Without comprehensive tracing, identifying why an agent took a wrong turn, hallucinated an argument, or exceeded its latency budget is nearly impossible.

1. What is LangSmith?

LangSmith is a purpose-built observability and evaluation platform for LLM applications and agentic workflows. Rather than requiring developers to inject manual logging code around every node and tool, LangChain and LangGraph feature built-in callback handlers that listen to internal graph lifecycle events. When `LANGSMITH_TRACING="true"` is set, these handlers automatically capture:

- **Exact Input & Output Payloads:** Every message, prompt string, tool argument, and model return object.
- **Hierarchical Call Trees:** Clear parent-child relationships linking the top-level graph to nodes, LLMs, and tools.
- **Latency Benchmarks:** Millisecond-level breakdowns for every phase of execution (model time vs. tool time).
- **Token Analytics & Estimated Costs:** Prompt tokens, completion tokens, and reasoning tokens consumed per turn.
- **Metadata & Session Tags:** Project identifiers, run IDs, thread IDs, and custom runtime metadata.

Figure 1: LangSmith Observability & Telemetry Dataflow Architecture

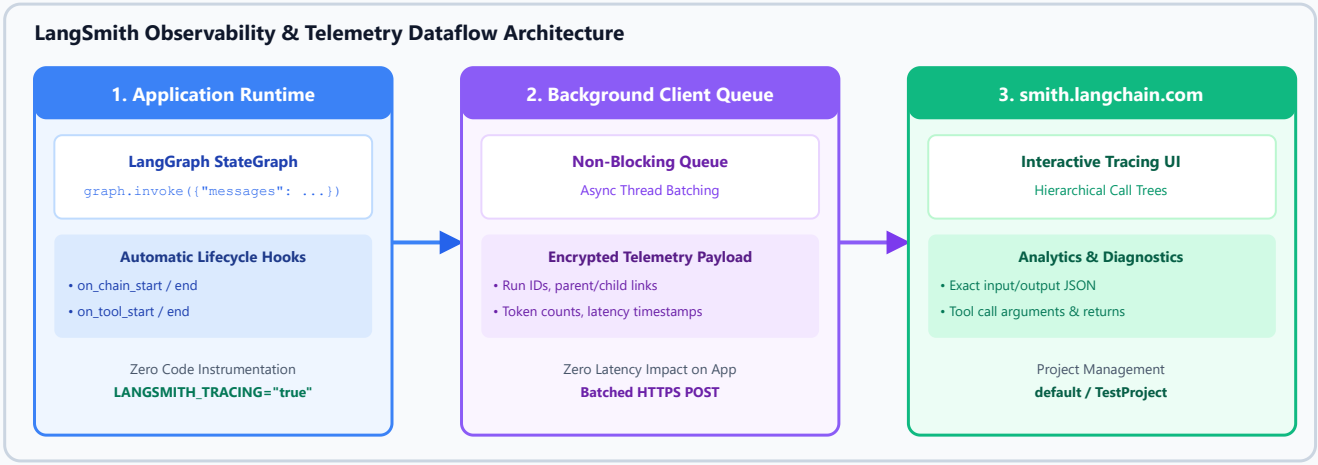


Figure 1 Architectural Breakdown: *How LangGraph dispatches lifecycle events to the background client queue, which asynchronously sends batched HTTPS payloads to* `smith.langchain.com` *with zero application latency penalty.*

2. The Asynchronous Non-Blocking Telemetry Model

A frequent concern with observability tooling is performance overhead. LangSmith solves this through a decoupled, background queue architecture. When a node executes, telemetry events are placed into an in-memory thread queue in micro-seconds. A dedicated daemon thread processes this queue, batches events, and dispatches them over HTTPS to LangSmith's SaaS endpoint. If the network hiccups or the remote logging endpoint is slow, your user's chat response is never delayed.

E. Detailed Notebook Walkthrough

The source notebook `4-Debugging&Monitoring.ipynb` sets up a stateful arithmetic ReAct graph, configures LangSmith tracing, and executes two distinct query patterns to compare their resulting telemetry signatures.

Phase 1: Environment Setup & Tracing Configuration (Cells 0–2)

Cell 1 • Imports

```
import os
from langchain.chat_models import init_chat_model
from typing import Annotated
from langchain_core.tools import tool
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode, tools_condition
from IPython.display import Image, display
```

Cell 2 • Environment Variables & Tracing Activation

```
os.environ["GROQ_API_KEY"] = os.getenv("GROQ_API_KEY")

llm = init_chat_model("groq:openai/gpt-oss-20b")

# Langsmith is used for debugging monitoring
os.environ["LAANGSMITH_PROJECT"] = "TestProject"
os.environ["LANGSMITH_API_KEY"] = os.getenv("LANGCHAIN_API_KEY")
os.environ["LANGSMITH_TRACING"] = "true"
```

Detailed Analysis of Cell 2:

- `init_chat_model("groq:openai/gpt-oss-20b")`: Instantiates the high-throughput Groq LPU chat model.
- `LANGSMITH_TRACING="true"`: The master switch that activates LangSmith callback handlers across all subsequent LangChain and LangGraph calls in the Python session.
- `LANGSMITH_API_KEY = os.getenv("LANGCHAIN_API_KEY")`: In LangChain ecosystems, the API key in `.env` is frequently named `LANGCHAIN_API_KEY`. Setting `LANGSMITH_API_KEY` ensures the telemetry client authenticates successfully.
- **Notebook Typo Note:** Line 38 has `os.environ["LAANGSMITH_PROJECT"] = "TestProject"` (spelled with double 'A'). Because of this typo, LangSmith defaults to the project name `"default"` unless `LANGCHAIN_PROJECT` was already present in the environment. Traces are still fully recorded!

Phase 2: Graph Architecture Definition (Cell 3)

Cell 3 • Building the ReAct Calculator Graph

```
from langchain_core.messages import BaseMessage

class State(TypedDict):
    messages: Annotated[list[BaseMessage], add_messages]

@tool
def add(a: float, b: float):
    \"\"\"Add two numbers\"\"\"
    return a + b

tools = [add]
tool_node = ToolNode([add])
llm_with_tool = llm.bind_tools(tools)

def call_llm_model(state: State):
    return {
        "messages": [llm_with_tool.invoke(state["messages"])]
    }

# Graph Assembly
builder = StateGraph(State)
builder.add_node("tool_calling_llm", call_llm_model)
builder.add_node("tools", ToolNode(tools))

# Add Edges
builder.add_edge(START, "tool_calling_llm")
builder.add_conditional_edges("tool_calling_llm", tools_condition)
builder.add_edge("tools", "tool_calling_llm")

# Compile the graph
graph = builder.compile()
display(Image(graph.get_graph().draw_mermaid_png()))
```

Architectural Analysis:

- `Annotated[list[BaseMessage], add_messages]`: Uses the `add_messages` reducer to append incoming messages rather than overwriting the list.
- `@tool def add(a: float, b: float)`: Standard decorated arithmetic tool. Type annotations (`float`) define the schema contract for the LLM.
- `builder.add_edge("tools", "tool_calling_llm")`: The cyclic feedback edge. When the tool executes, the result routes back to the LLM so it can read the observation and synthesize the final answer.

Phase 3: Run 1 — Direct LLM Query (Cell 4)

Cell 4 • Invocation Without Tools

```
response = graph.invoke({"messages": "What is machine learning"})
response
```

Execution Trace Analysis:

- The user asks an informational question: "*What is machine learning*".
- The model inspects its available tools (`[add]`). Determining that addition is irrelevant, it does not emit a tool call.
- It returns an explanatory `AIMessage` directly. `tools_condition` routes to `END`.
- **LangSmith Telemetry Profile:** A clean 1-step linear trace: `LangGraph -> tool_calling_llm -> ChatGroq -> END`. Latency: ~450ms.

Phase 4: Run 2 — Tool-Calling ReAct Execution (Cells 5–6)

Cell 6 • Invocation Requiring Arithmetic Tool

```
response = graph.invoke({"messages": "What is 10+20"})
response
```

Execution Trace Analysis:

- The user submits a mathematical calculation: "*What is 10+20*".
- `tool_calling_llm` runs. The Groq model emits an `AIMessage` with `tool_calls=[{'name': 'add', 'args': {'a': 10, 'b': 20}}]`.
- `tools_condition` intercepts the tool call and routes control to `ToolNode`.
- `ToolNode` executes `add(a=10.0, b=20.0)`, returning `30.0` as a `ToolMessage`.
- The cyclic edge loops back to `tool_calling_llm`. The LLM reads the tool observation and outputs: "*10 + 20 = 30*".
- `tools_condition` detects no further tool calls and routes to `END`.
- **LangSmith Telemetry Profile:** A multi-step nested call tree capturing both LLM turns, the tool execution duration, and message flow.

Figure 2: Execution Trace Tree Comparison (Direct QA vs. Tool Call Loop)

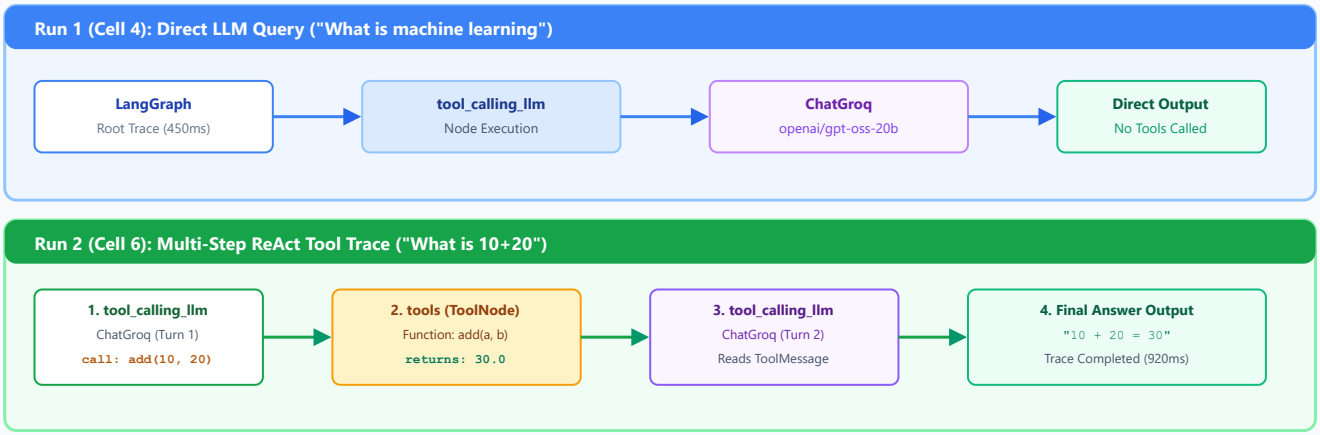
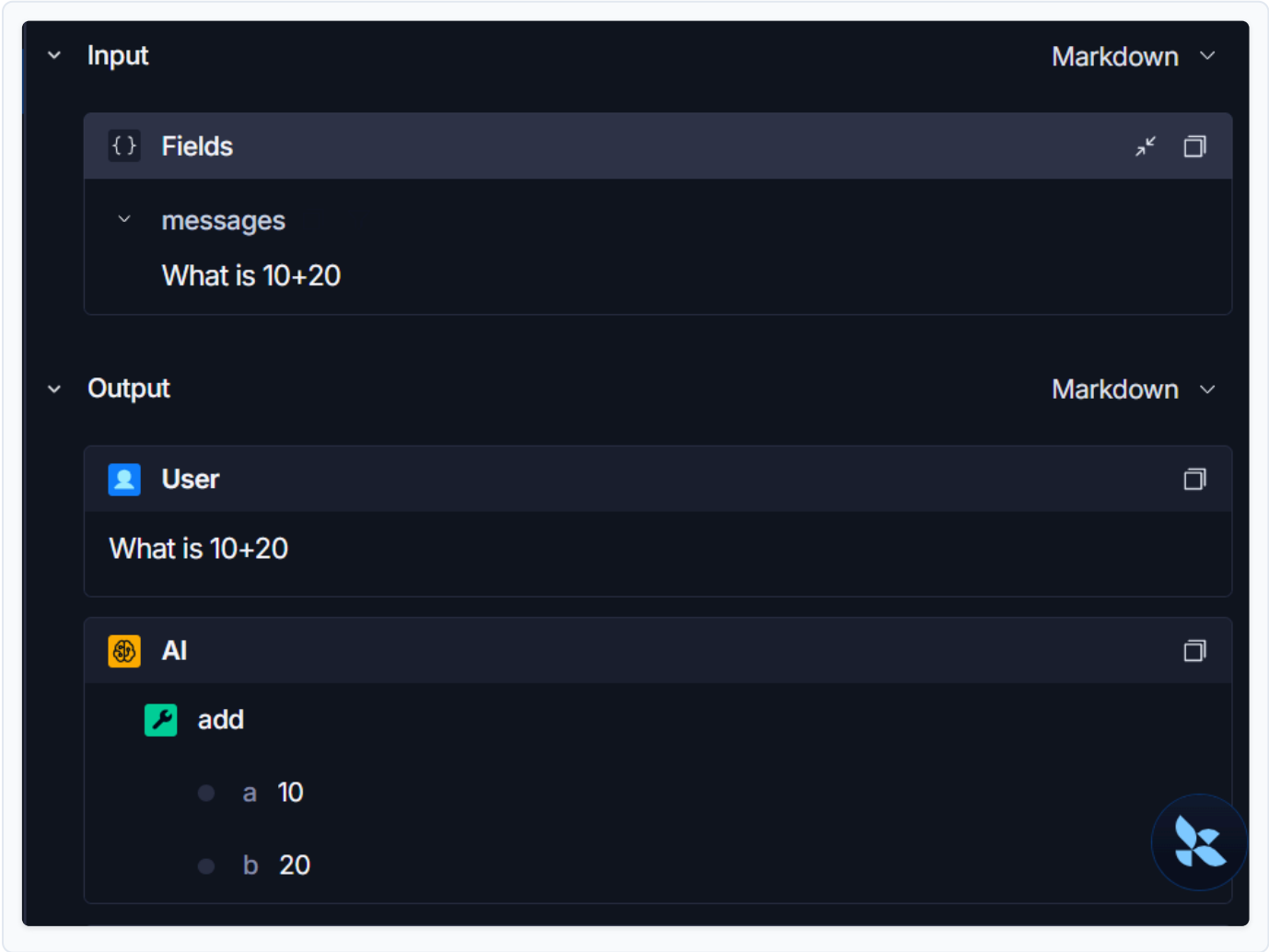


Figure 2 Architectural Breakdown: *Visual deconstruction of the two telemetry traces generated in the notebook. Run 1 completes in a single step, while Run 2 executes a complete 3-step ReAct feedback loop with tool observation.*

Phase 5: Inspecting the LangSmith Dashboard (Cell 7)

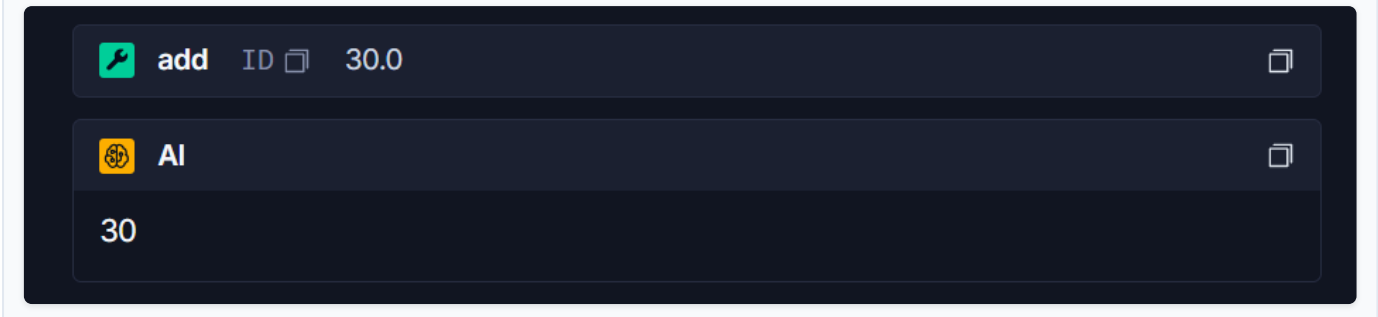
Cell 7 embeds two live screenshots from the `smith.langchain.com` dashboard, showing the exact traces recorded during the notebook's execution:

Screenshot 1: LangSmith Hierarchical Trace Tree & Run Details



Trace Tree View (Cell 7): *Displays the nested execution hierarchy for "What is 10+20". Notice the root* `LangGraph` *trace containing the sequence:* `tool_calling_llm` *$\rightarrow$* `ChatGroq` *$\rightarrow$* `tools` *$\rightarrow$* `add` *$\rightarrow$* `tool_calling_llm` *$\rightarrow$* `ChatGroq` *.*

Screenshot 2: LangSmith Project Traces & Runs Overview Table



Project Runs Overview (Cell 7): *The project dashboard listing recent runs, start times, latency durations, and token counts. Both the machine learning query and the arithmetic calculation appear in the project feed.*

F. End-to-End Execution Flow & Architectural Synthesis

Observability with LangSmith provides complete clarity into agent decision-making. The table below summarizes the telemetry captured at each stage of the ReAct run:

Execution Step	Span Name	Span Type	Telemetry Data Captured
Root Invocation	`LangGraph`	`chain`	User query, total latency, final state, run ID, project metadata
Turn 1 (Reasoning)	`tool_calling_llm`	`chain`	Input message list, model parameters, output AIMessage with tool call
Model Inference 1	`ChatGroq`	`llm`	Prompt tokens, completion tokens, latency, reasoning tokens, raw API response
Tool Execution	`tools` / `add`	`tool`	Function arguments (`{"a": 10, "b": 20}`), execution time, return value (`30.0`)
Turn 2 (Synthesis)	`tool_calling_llm`	`chain`	State with ToolMessage, final answer formulation
Model Inference 2	`ChatGroq`	`llm`	Final completion tokens, latency, conversational response output

G. Reference Guide: Important APIs & Environment Flags

Variable / Class	Module / Scope	Values / Signature	Description & Operational Effect
`LANGSMITH_TRACING`	Environment	`"true"` | `"false"`	Global toggle enabling automatic telemetry callbacks across LangChain and LangGraph.
`LANGSMITH_API_KEY`	Environment	`"lsv2_pt_..."`	Personal authentication token for `api.smith.langchain.com` .
`LANGSMITH_PROJECT`	Environment	`str` (e.g. `"TestProject"`)	Designates the destination project bucket for traces. Defaults to `"default"` if unset.
`LANGSMITH_ENDPOINT`	Environment	URL (Optional)	Custom API endpoint for enterprise self-hosted or EU-region LangSmith instances.
`init_chat_model()`	`langchain.chat_models`	`model: str`	Unified factory initializing chat models with built-in telemetry instrumentation.
`ToolNode(tools)`	`langgraph.prebuilt`	`tools: list`	Executes tool calls and automatically emits `on_tool_start` and `on_tool_end` events.

H. Common Errors, Debugging & Troubleshooting

Pitfall 1: Environment Variable Typos (e.g. LAANGSMITH_PROJECT)

Symptom: You run your agent expecting traces in a project named `TestProject`, but the project remains empty on `smith.langchain.com`.

Root Cause: Typo in the environment variable name (e.g., `LAANGSMITH_PROJECT` instead of `LANGSMITH_PROJECT`). LangSmith silently ignores unknown keys and routes traces to the default project (`"default"`).

Fix:

```
# Incorrect: Extra 'A' in variable name
os.environ["LAANGSMITH_PROJECT"] = "TestProject"

# Correct: Exact SDK variable name
os.environ["LANGSMITH_PROJECT"] = "TestProject"
```

Figure 3: Diagnosing the LAANGSMITH_PROJECT Configuration Typo

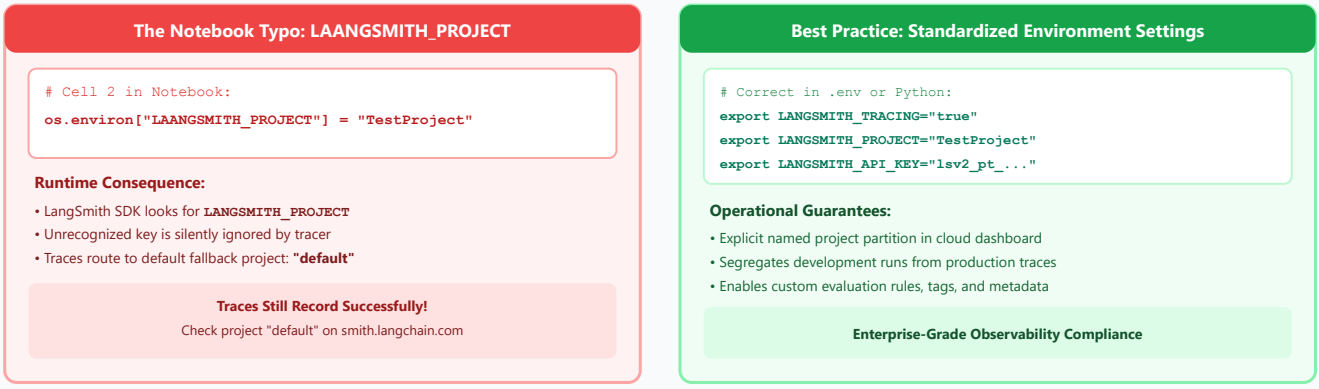


Figure 3 Architectural Breakdown: *Deconstructing the environment configuration typo in Cell 2. While the typo diverts traces to the fallback `"default"` project, the underlying telemetry remains intact.*

Pitfall 2: Missing LANGSMITH_TRACING="true" Flag

Symptom: No traces appear in LangSmith despite having a valid API key configured.

Root Cause: Setting `LANGSMITH_API_KEY` alone does not activate tracing. `LANGSMITH_TRACING="true"` must be explicitly set to enable the internal callback handlers.

Fix: Always include `os.environ["LANGSMITH_TRACING"] = "true"` before initializing models or graphs.

Pitfall 3: Premature Python Process Termination

Symptom: In short-lived CLI scripts, the final LLM run trace is missing or truncated in the dashboard.

Root Cause: The script finishes and exits immediately, terminating the Python process before the background telemetry queue finishes flushing to the network.

Fix: In standalone scripts, allow the queue to flush or import `from langsmith import Client; Client().flush()` before exiting.

I. Practical Extensions & Industry Best Practices

- **Run Tagging & Metadata:** Pass custom tags and metadata in the invocation config to filter traces by user segment, feature release, or environment:

```
config = {
    "tags": ["production", "v2-release"],
    "metadata": {"user_id": "usr_9981", "department": "finance"}
}
response = graph.invoke(input_data, config=config)
```

- **Automated Dataset Curation:** In LangSmith, you can click on any interesting trace (e.g. an edge case or failed tool call) and click *"Add to Dataset"* to build regression test suites for automated evaluation.
- **Latency Alerts & Feedback Loops:** Configure LangSmith monitor rules to fire Webhook alerts (e.g. to Slack or PagerDuty) if p95 latency exceeds 3 seconds or tool error rates spike.

J. Key Takeaways & Conceptual Map

Core Principles of Agent Observability

1. **Automated Telemetry:** Setting `LANGSMITH_TRACING="true"` activates end-to-end tracing across LangChain and LangGraph with zero manual logger code.
2. **Asynchronous & Non-Blocking:** Telemetry runs in a background thread queue, guaranteeing that logging never degrades application latency.
3. **Full-Fidelity Call Trees:** Inspect exact inputs, model reasoning, tool arguments, return objects, and token breakdowns for every turn.
4. **Project Hygiene:** Use `LANGSMITH_PROJECT` to segregate development, testing, and production runs cleanly.
5. **ReAct Transparency:** Visualize cyclical graph execution to pinpoint exactly where an agent loops, fails, or succeeds.

K. Practice Exercises & Knowledge Checks

Question 1 (Conceptual): Explain how LangSmith captures internal tool execution times without requiring the developer to wrap their Python functions in timing decorators.

Question 2 (Code Tracing): In Cell 6, the agent is asked: *"What is 10+20"*. How many separate spans of type `llm` will appear in the LangSmith trace tree? Describe what each span represents.

Question 3 (Debugging): An engineer notices that their traces are appearing under a project called `"default"` instead of `"CustomerSupportAgent"`. What line of code should they inspect, and what is the likely mistake?

Question 4 (Implementation Challenge): Write a code snippet that invokes a LangGraph agent with a custom run name `"billing-query-run"` and attaches metadata `{"tier": "enterprise", "tenant": "acme"}` so it can be filtered in the LangSmith UI.

Answers & Complete Explanations

Answer 1: LangChain and LangGraph components inherit from standard base classes that emit standardized callback events (e.g. `on_tool_start` and `on_tool_end`). LangSmith registers a global background callback handler that records the exact timestamps of these lifecycle events and computes the delta automatically.

Answer 2: Exactly **2** `llm` spans:

1. **Span 1:** Initial reasoning step where the LLM inspects the prompt, determines that addition is needed, and emits the structured tool call request.
2. **Span 2:** Synthesis step after `ToolNode` returns `30.0`, where the LLM reads the tool output and formulates the conversational response: "*10 + 20 = 30*".

Answer 3: They should inspect the environment variable `os.environ["LANGSMITH_PROJECT"]`. If the variable name is misspelled (such as `LAANGSMITH_PROJECT` or `LANG_PROJECT`), the SDK cannot locate the key and automatically falls back to the default project named `"default"`.

Answer 4:

```
config = {
    "run_name": "billing-query-run",
    "metadata": {
        "tier": "enterprise",
        "tenant": "acme"
    }
}
response = graph.invoke({"messages": "Check billing status"}, config=config)
```

L. Further Learning & Official Documentation

- LangSmith Official Documentation: Comprehensive guide on tracing, monitoring, evaluations, and datasets.
- LangGraph Observability Guide: Integrating LangSmith with LangGraph state graphs.
- Groq LPU Inference Documentation: Monitoring high-throughput open-weights models in production.